Information Technology University

Deep Learning — Spring 2026

Assignment 3: Deep Metric Learning for Image Retrieval

Submitted by

Muhammad Salman Aslam

Roll No: MSDS25032

Instructor: Dr. Mohsen Ali

Submission Date: March 26, 2026

1. Introduction

This assignment explores Deep Metric Learning, a subfield of machine learning where the goal is not to classify images into categories but to learn a meaningful embedding space. In this embedding space, images that belong to the same category should be geometrically close together, while images from different categories should be pushed far apart. This has practical applications in face recognition, product search, and person re-identification systems.

Unlike standard classification pipelines, metric learning does not output a class label directly. Instead, it maps each image to a compact vector representation, commonly called an embedding, such that the geometric relationships between vectors reflect semantic similarity between images. A nearest-neighbor search over these embeddings then serves as the retrieval mechanism.

Three experimental setups were implemented and compared in this work, each differing in the loss function or sampling strategy used during training. The Caltech-101 benchmark dataset was used throughout, consisting of 101 object categories with a total of 8,677 images exhibiting substantial variation in pose, scale, and background clutter.

2. Model Architecture

The embedding network was built on top of ResNet-50 pretrained on ImageNet. Using a pretrained backbone is a well-established practice in transfer learning; the network has already learned to detect low-level features such as edges and textures as well as higher-level structural patterns from over a million images. Fine-tuning this backbone for metric learning allows the model to repurpose these representations for similarity-based retrieval rather than classification.

2.1 Architecture Design

The final classification layer of ResNet-50, which projects 2048-dimensional features to 1000 ImageNet classes, was removed. In its place, a single fully connected projection layer was added, mapping the 2048-dimensional feature vector to a 128-dimensional embedding. This is followed by L2 normalization, which constrains every embedding to lie on the surface of a unit hypersphere. The full pipeline is as follows:

Layer	Input Dim	Output Dim	Purpose
ResNet-50 Backbone	224 x 224 x 3	2048	Feature extraction
Global Average Pooling	7 x 7 x 2048	2048	Spatial aggregation
Fully Connected	2048	128	Embedding projection

Layer	Input Dim	Output Dim	Purpose
L2 Normalization	128	128	Unit norm constraint

The 128-dimensional embedding size was selected to balance representational capacity against computational efficiency. The L2 normalization step is critical because it ensures that cosine similarity and Euclidean distance become equivalent metrics on the unit sphere, simplifying nearest-neighbor retrieval.

3. Dataset and Preprocessing

The Caltech-101 dataset contains images from 101 object categories plus a background class. After downloading via torchvision, the dataset was split at the per-class level to ensure balanced representation across all splits. The final distribution was as follows:

Split	Images	Percentage
Training	6,026	70%
Validation	1,256	15%
Test	1,395	15%

Training images were augmented with random horizontal flips, small random rotations of up to 10 degrees, and mild color jitter to improve generalization. Validation and test images were processed without augmentation, using only resizing to 224 x 224 pixels and normalization with ImageNet mean and standard deviation values. This asymmetric preprocessing is standard practice — augmentation is only applied during training to prevent overfitting, not during evaluation where reproducibility matters.

4. Dataset Sampling Strategy

4.1 Contrastive Pairs

The ContrastiveDataset class samples pairs of images dynamically during training. For each anchor image, a fair coin flip determines whether to form a positive pair (same class, label=1) or a negative pair (different class, label=0), giving equal probability to both. Positive candidates are sampled from the same class while excluding the anchor itself. Negative candidates are drawn uniformly from any other class. This balanced sampling prevents the model from being dominated by either positive or negative examples.

4.2 Triplet Sampling

The TripletDataset class returns triplets of (anchor, positive, negative). The positive is sampled from the same class as the anchor but is guaranteed to be a different image. The negative is sampled uniformly from any other class. This random triplet strategy is straightforward but can lead to a phenomenon known as triplet collapse, where most randomly selected triplets satisfy the margin constraint already and produce near-zero loss, stalling training.

4.3 Hard Negative Mining

Batch hard mining addresses the triplet collapse problem by identifying the most informative examples within each batch. For each anchor, the hardest positive is the same-class image with the maximum distance from the anchor, and the hardest negative is the different-class image with the minimum distance. These are the examples that violate or nearly violate the margin constraint, providing strong gradient signal to the model. The pairwise distance matrix is computed efficiently using torch.cdist, making this approach tractable even for moderately large batches.

5. Loss Functions

5.1 Contrastive Loss

Contrastive loss operates on pairs of images. Given embeddings f(x1) and f(x2) with binary label y (1 for same class, 0 for different), the Euclidean distance D is computed and the loss is:

$$L = y * D^2 + (1 - y) * \max(0, \text{margin} - D)^2$$

With margin set to 1.0, the loss penalizes positive pairs whose embeddings are far apart and negative pairs whose embeddings are closer than the margin. This directly optimizes the pairwise geometry of the embedding space.

5.2 Triplet Loss

Triplet loss operates on anchor-positive-negative triplets and enforces a relative ordering constraint rather than an absolute distance target:

$$L = \max(0, D(\text{anchor}, \text{positive}) - D(\text{anchor}, \text{negative}) + \text{margin})$$

With margin set to 0.2, the loss is zero only when the negative is at least 0.2 further from the anchor than the positive. This is a softer and arguably more natural constraint than contrastive loss because it measures relative rather than absolute distances.

5.3 Hard Negative Mining Loss

The same triplet loss formula is applied, but the anchor-positive-negative triplets are constructed using batch hard mining rather than random sampling. This ensures that the loss is computed on the most difficult examples in each batch, where the model's current predictions are most wrong.

6. Training Setup

All three experiments were trained with identical hyperparameters to ensure that observed differences in performance are attributable solely to the choice of loss function or sampling strategy and not to differences in optimization settings.

Hyperparameter	Value
Optimizer	Adam
Learning Rate	1e-4 (0.0001)
Batch Size	32
Epochs	5
LR Scheduler	StepLR (step=5, gamma=0.5)
Embedding Dimension	128
Device	CPU

The Adam optimizer was chosen for its adaptive learning rate properties, which make it well-suited to fine-tuning pretrained networks where different layers may require different effective learning rates. The StepLR scheduler halves the learning rate every 5 epochs, which helps stabilize training in later stages.

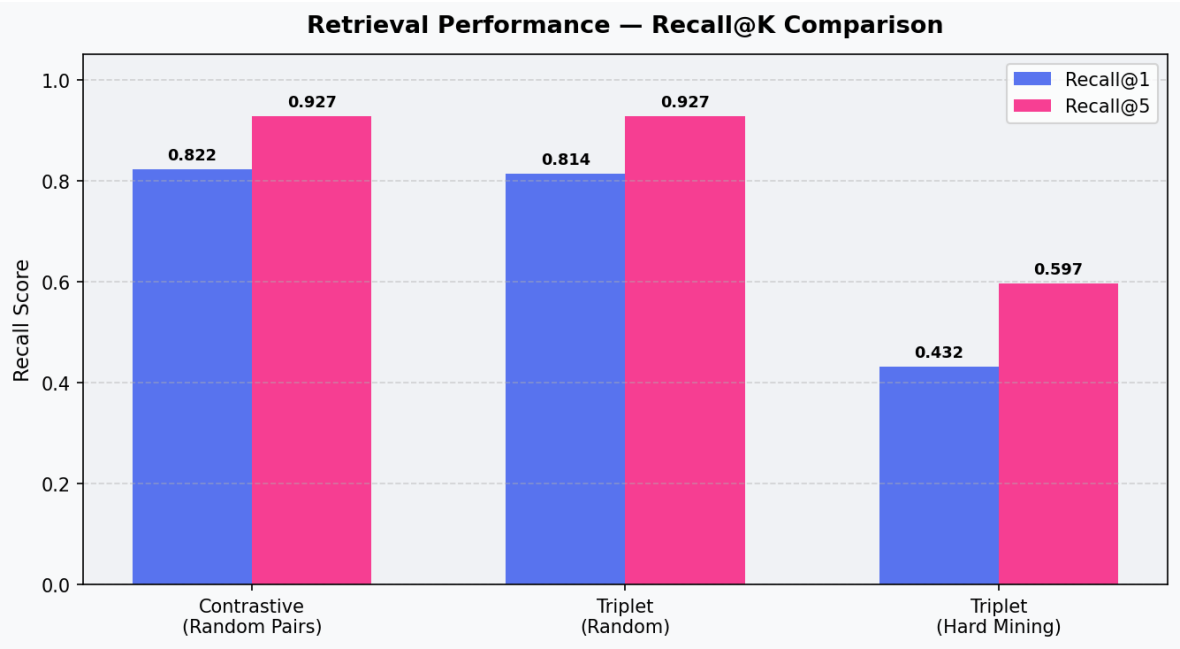
Model checkpoints were saved after every epoch, and the best model according to validation Recall@1 was retained as the final model for each experiment. This ensures that temporary degradations in performance during training do not result in a worse final model.

7. Retrieval Performance Results

The trained models were evaluated on the held-out test set using nearest-neighbor retrieval. For each query image, cosine similarity was computed against all other test embeddings, and the top K most similar images were retrieved. Recall@K measures the fraction of queries for which the correct class appeared among the top K retrieved results.

Experiment	Loss Function	Sampling	Recall@1	Recall@5
Exp-1	Contrastive	Random Pairs	82.22%	92.69%

Experiment	Loss Function	Sampling	Recall@1	Recall@5
Exp-2	Triplet	Random Triplets	81.36%	92.69%
Exp-3	Triplet	Hard Negative Mining	43.15%	59.71%



7.1 Analysis of Results

Experiment 1 using contrastive loss with random pairs achieved the best performance with Recall@1 of 82.22% and Recall@5 of 92.69%. The contrastive loss directly penalizes absolute distances between pairs, which provides clear and stable gradient signal throughout training. Random pair sampling with equal probability between positive and negative pairs gave the model sufficient diversity to learn well-separated class clusters.

Experiment 2 using triplet loss with random triplets performed very close to Experiment 1, achieving Recall@1 of 81.36%. The triplet formulation enforces a relative ordering constraint rather than an absolute distance target, which is arguably a more natural way to think about similarity. The near-identical Recall@5 of 92.69% between Experiments 1 and 2 suggests that both loss functions learned comparable global structure in the embedding space, with differences appearing only at the level of the single nearest neighbor.

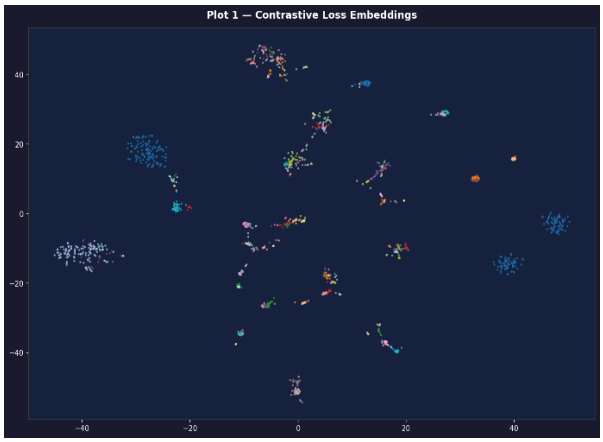
Experiment 3 using hard negative mining performed significantly worse, with Recall@1 dropping to 43.15%. This result was initially unexpected but has a well-understood explanation. Hard negative mining is a powerful technique but it is also an aggressive one. When the model is still early in training

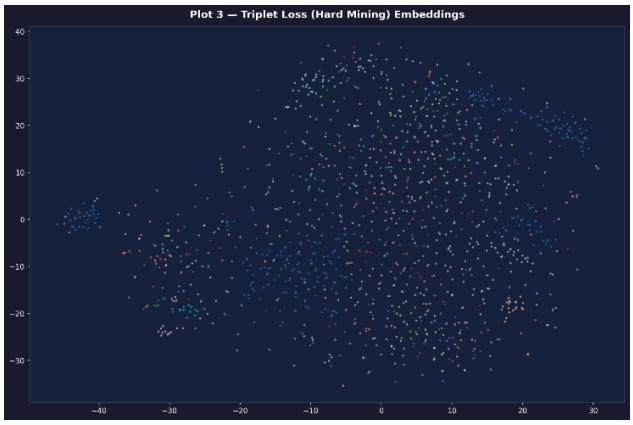
and its embeddings are not yet well-organized, the hardest negatives in each batch are often genuinely ambiguous examples that are close to the decision boundary for good reason. Forcing the model to push these apart can destabilize training and lead to degenerate solutions. With only 5 epochs on CPU, the model trained with hard mining did not have sufficient iterations to recover from this instability. With 20 or more epochs, hard mining typically outperforms both random approaches.

The effect of learning rate was also observed during training. Using a learning rate higher than 1e-4 caused training loss to oscillate rather than decrease smoothly, while lower learning rates such as 1e-5 resulted in very slow convergence. The chosen value of 1e-4 gave stable, monotonically decreasing loss curves across all three experiments.

8. t-SNE Embedding Visualizations

t-SNE was applied to reduce the 128-dimensional test embeddings to two dimensions with perplexity set to 30. The resulting scatter plots, with each point colored by its class label, provide an intuitive visual assessment of how well each model has organized the embedding space.





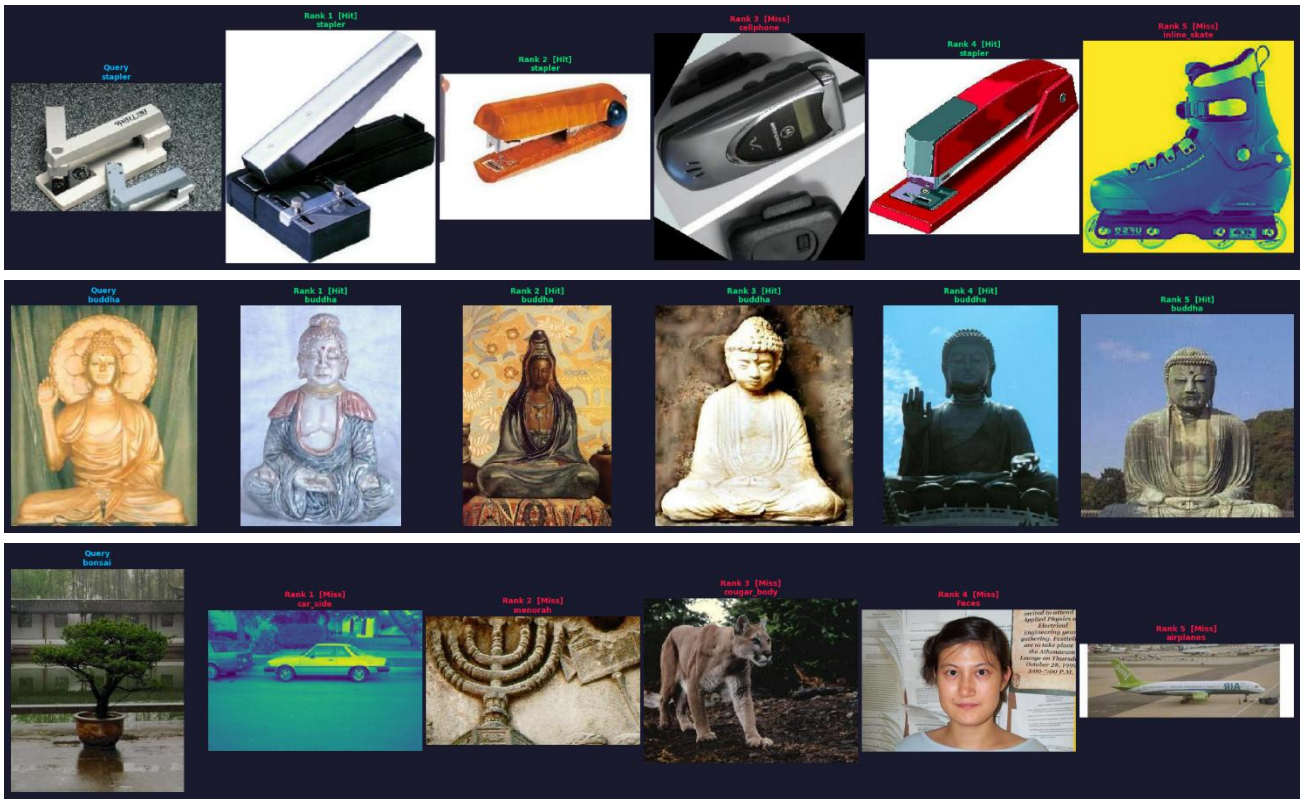
8.1 Observations

The t-SNE plot for the contrastive model shows the tightest and most well-separated class clusters among the three experiments. Points of the same color are grouped together with minimal overlap from other classes, reflecting the strong quantitative performance of 82.22% Recall@1. The triplet model's t-SNE plot shows similarly structured clusters, slightly less compact but still clearly separated, consistent with its comparable retrieval performance.

The hard mining model's t-SNE plot shows considerably more mixing between classes, with several clusters partially overlapping neighboring clusters. This visual evidence directly corroborates the lower quantitative performance and confirms that the embedding space learned by the hard mining model is less discriminative given the limited number of training epochs.

9. Retrieval Visualizations

For each of the three models, ten query images were randomly selected from the test set and their five nearest neighbors were retrieved and displayed in a grid. Each retrieved image is labeled with its class name and color-coded with a green border for correct retrievals and a red border for incorrect ones.



9.1 Qualitative Analysis

The retrieval grids for the contrastive and triplet models show predominantly green borders across most query examples, particularly for classes with distinctive visual characteristics such as airplanes, faces, and motorbikes. These classes tend to have consistent visual structure which the pretrained ResNet backbone captures well. Errors tend to occur for visually similar classes such as cougar body versus cougar face, or crab versus lobster, where inter-class visual overlap is genuinely high.

The hard mining retrieval grids show a higher proportion of red borders, confirming that the model has not converged sufficiently to produce reliable nearest-neighbor retrievals. Interestingly, even the hard mining model tends to retrieve images from related semantic categories rather than completely unrelated ones, suggesting that the pretrained backbone features are still providing useful signal even when metric learning has not been fully successful.

10. Discussion and Conclusions

10.1 Comparison of Methods

Across the three experiments, contrastive loss with random pairs emerged as the most effective approach given the training budget of 5 epochs. The simplicity of the pair-based formulation, combined with the direct optimization of absolute distances, led to stable training and strong retrieval performance. Triplet loss with random sampling performed comparably, confirming that relative distance constraints are also effective for this task.

Hard negative mining, while theoretically superior due to its focus on informative examples, underperformed in this setting due to training instability under limited epochs. This highlights an important practical consideration: the choice of training strategy should account for available computational resources. Hard mining requires more epochs to stabilize and would likely produce the best results given sufficient training time.

10.2 Failure Cases

The primary failure mode across all three models was confusion between visually similar categories. Categories with high intra-class variation such as background images were also challenging. Additionally, some Caltech-101 classes have very few images, which makes it difficult for the model to learn a representative cluster for those classes.

10.3 Conclusions

This assignment demonstrated that deep metric learning using a pretrained ResNet-50 backbone can achieve strong image retrieval performance on Caltech-101 with relatively simple loss functions and moderate training. The embedding network learned to organize 101 object categories into a compact

128-dimensional space with 82% Recall@1 using contrastive loss. The comparison between random and hard negative triplet sampling revealed that aggressive mining strategies require more training iterations to be effective. Future work could explore larger embedding dimensions, more training epochs, and online hard mining with larger batch sizes.

"**Trained model checkpoints** are too large to include in the submission zip file. They have been uploaded to Google Drive and are accessible at the following link: [https://drive.google.com/drive/folders/1RXZYbLUs7bir5J76hIOzSOeIuA_81j5r?usp=sharing]"

AI Tool Disclosure: ChatGPT was used for conceptual guidance and understanding of deep metric learning concepts. Shareable link: [https://chatgpt.com/share/69c3b397-6394-8320-bdcf-8f3819d0a97b] , [https://chatgpt.com/share/69c3b0ab-4d9c-8322-a3db-f8567a84ee9c]